LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL 5 – Relasi Antar Kelas



RAYMUNDUS ARIEL ABAS

103132430021

S1SD-05-01

PROGRAM STUDI S-1 SAINS DATA

FAKULTAS DIREKTORAT KAMPUS PURWOKERTO

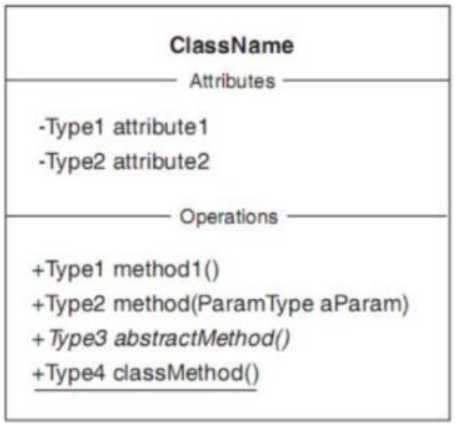
TELKOM UNIVERSITY

2026

A. Dasar Teori

1. Diagram Kelas / *Class Diagram*

Diagram kelas (*Class Diagram*) merupakan salah satu diagram dalam Unified Modeling Language (UML) yang digunakan untuk menggambarkan struktur suatu sistem berdasarkan kelas-kelas yang terdapat di dalamnya beserta hubungan antar kelas. Diagram kelas terdiri atas tiga bagian utama, yaitu nama kelas, atribut yang merepresentasikan data atau karakteristik kelas, serta method yang merepresentasikan fungsi atau perilaku kelas.



Selain itu, diagram kelas menggunakan *access modifier* untuk menentukan tingkat akses terhadap atribut dan method, yang meliputi *public* (+), *private* (-), dan *protected* (#).

Simbol	Keterangan
+	simbol *access modifier public*
-	simbol *access modifier private*
#	simbol *access modifier protected* (untuk simbol ini tidak terdapat pada gambar diatas)

2. Relasi /Hubungan Antar-Kelas

Konsep *Object-Oriented Programming* (OOP) memungkinkan setiap kelas saling terhubung sesuai kebutuhan dan logika sistem. Relasi antar-kelas ini terbagi ke dalam beberapa bentuk, di antaranya:

a. Asosiasi

Asosiasi mendefinisikan keterhubungan struktural antara dua kelas independen yang berinteraksi melalui masing-masing objeknya. Ketika dua kelas perlu saling bertukar informasi atau berkomunikasi, keterkaitan tersebut direpresentasikan melalui relasi asosiasi.

Notasi asosiasi digambarkan pada diagram berikut:

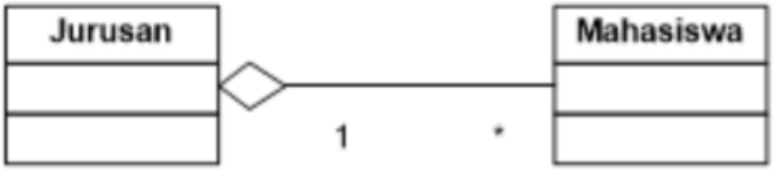


Diagram di atas menunjukkan keterikatan antara kelas Dosen dan kelas Mahasiswa. Relasi asosiasi tercipta karena kelas Dosen mengakses atau memanggil *method* yang didefinisikan di dalam kelas Mahasiswa.

b. Agregasi

Agregasi merupakan relasi struktural antar-kelas yang menggambarkan keterikatan *has-a* dengan konsep bagian (*part*) dan keseluruhan (*whole*). Hubungan ini bersifat lebih mengikat dibandingkan asosiasi umum, namun siklus hidup (*lifecycle*) objek bagian tidak bergantung penuh pada objek induknya. Secara implementasi, agregasi diwujudkan dengan menjadikan suatu kelas sebagai tipe atribut pada kelas lainnya.

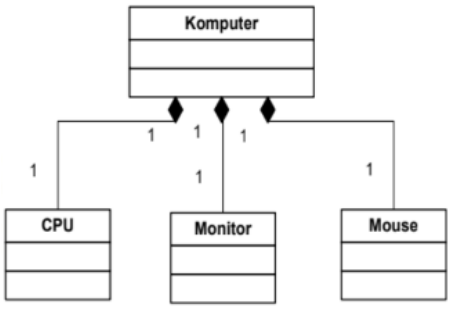
Notasi relasi ini disimbolkan sebagai berikut:



Pada ilustrasi di atas, relasi agregasi tercermin melalui kelas Jurusan yang memiliki atribut bertipe kelas Mahasiswa. Hal ini menandakan bahwa kumpulan objek Mahasiswa merupakan bagian dari Jurusan, namun entitas Mahasiswa secara konseptual tetap dapat berdiri sendiri.

c. Komposisi

Komposisi adalah varian khusus dari agregasi dengan derajat keterikatan (*coupling*) yang paling kuat. Karakteristik utama komposisi terletak pada ketergantungan siklus hidup: keberadaan kelas bagian (*part*) sepenuhnya terikat pada kelas induk (*whole*).



Berdasarkan diagram tersebut, kelas CPU, Monitor, dan Mouse berperan sebagai komponen pembentuk dari kelas Komputer. Apabila objek Komputer dihapus atau dihancurkan (*destroyed*), seluruh komponen di dalamnya (CPU, Monitor, dan Mouse) secara otomatis ikut tereliminasi karena tidak dapat eksis secara terpisah tanpa induknya.

B. Guided

1. Asosiasi

a. Dosen.java

```
package raymundusariel.guided.guided1;


public class Dosen {


   private String kodeDosen;
   private String namaDosen;


   // Setter
   public void setKodeDosen(String kodeDosen) {
      this.kodeDosen = kodeDosen;
   }


   public void setNamaDosen(String namaDosen) {
      this.namaDosen = namaDosen;
   }


   // Getter
   public String getKodeDosen() {
      return this.kodeDosen;
   }


   public String getNamaDosen() {
      return this.namaDosen;
```

```
}

public void giveScore(Mahasiswa s, int nilai) {
    // Asosiasi: method milik class Mahasiswa dipanggil di class Dosen,
    // tapi objek Mahasiswa tidak menjadi atribut dari class Dosen
    s.setNilai(nilai);
}

public int getScore(Mahasiswa s) {
    // Asosiasi: method milik class Mahasiswa dipanggil di class Dosen,
    // tapi objek Mahasiswa tidak menjadi atribut dari class Dosen
    return s.getNilai();
}

public static void main(String[] args) {
    Mahasiswa m = new Mahasiswa();

    m.setNim("103132430021");
    m.setNama("Raymundus Ariel Abas");

    Dosen d = new Dosen();
    d.giveScore(m, 100);

    System.out.println("NIM :" + m.getNim());
    System.out.println("Nama :" + m.getNama());
```

```
            System.out.println("Nilai :" + d.getScore(m));

        }

    }
```

b. Mahasiswa.java

```
    package raymundusariel.guided.guided1;

    public class Mahasiswa {

        private String nim;
        private String nama;
        private int nilai;

        public void setNama(String nama) {
            this.nama = nama;
        }

        public void setNim(String nim) {
            this.nim = nim;
        }

        public void setNilai(int nilai) {
            this.nilai = nilai;
        }

        public int getNilai() {
            return this.nilai;
        }


        public String getNama() {
            return this.nama;
        }

        public String getNim() {
            return this.nim;
        }
}
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ guided ---
NIM :103132430021
Nama :Raymundus Ariel Abas
Nilai :100
```

2. Agregasi

a. Jurusan.java

```
package raymundusariel.guided.guided2;

public class Jurusan {

    private static int jumlahMahasiswa;

    private String kodeJurusan, namaJurusan;

    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];


    //Construcctor

    public Jurusan(String kodeJurusan, String namaJurusan) {

        this.kodeJurusan = kodeJurusan;

        this.namaJurusan = namaJurusan;

    }


    //Behaviour

    public void AddMahasiswa(Mahasiswa m) {

        this.daftarMahasiswa[jumlahMahasiswa] = m;

        jumlahMahasiswa++;

    }


    public void DisplayMahasiswa() {
```

```
        System.out.println("Kode Jurusan : " + this.kodeJurusan);
        System.out.println("Nama Jurusan : " + this.namaJurusan);
        System.out.println("Daftar Mahasiswa :");
        for (int i = 0; i < jumlahMahasiswa; i++) {
            System.out.println(i+1 + "." +daftarMahasiswa[i].getNIM() + " " +
daftarMahasiswa[i].getNama());
        }
    }

    //Getter
    public String getKodeJurusan() {
        return kodeJurusan;
    }

    public String getNamaJurusan() {
        return namaJurusan;
    }

    //Setter
     public void setKodeJurusan(String kodeJurusan) {
        this.kodeJurusan = kodeJurusan;
    }

    public void setNamaJurusan(String namaJurusan) {
        this.namaJurusan = namaJurusan;
```

```
    }
}
```

b. Mahasiswa.java

```
package raymundusariel.guided.guided2;

public class Mahasiswa {

    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama) {
        this.NIM = NIM;
        this.Nama = Nama;
    }

    //Getter
    public String getNIM() {
        return (NIM);
    }

    public String getNama() {
        return (Nama);
    }

     //Setter
```

```
    public void setNim(String NIM) {
        this.NIM = NIM;
    }

    public void setNama(String Nama) {
        this.Nama = Nama;
    }
}
```

3. Komposisi

a. Komputer.java

```
package raymundusariel.guided.guided3;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String
brandMonitor, String brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
```

```
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }

}
```

b. Monitor.java

```
package raymundusariel.guided.guided3;
public class Monitor {
    private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

c. Mouse.java

```
package raymundusariel.guided.guided3;
public class Mouse {
    private String brand, tipe;
```

```
    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

d. Processor.java

```
package raymundusariel.guided.guided3;
public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
```

```
    }
}
```

e. Main untuk guided 2 dan 3

```
package raymundusariel.guided;

import raymundusariel.guided.guided2.*;
import raymundusariel.guided.guided3.*;

public class Main {
    public static void main(String[] args) {
        runGuided2();
        runGuided3();
    }

    public static void runGuided2() {
        raymundusariel.guided.guided2.Mahasiswa m1 =
                new raymundusariel.guided.guided2.Mahasiswa(
                        "103132430021", "Raymundus Ariel Abas");

        raymundusariel.guided.guided2.Mahasiswa m2 =
                new raymundusariel.guided.guided2.Mahasiswa(
                        "103132430022", "Keyla Azzahra");

        raymundusariel.guided.guided2.Jurusan jur =
```

```
                new raymundusariel.guided.guided2.Jurusan(
                        "DS", "Data Science");


            jur.AddMahasiswa(m1);
            jur.AddMahasiswa(m2);


            jur.DisplayMahasiswa();
        }


        public static void runGuided3() {
            raymundusariel.guided.guided3.Komputer pc =
                    new raymundusariel.guided.guided3.Komputer(
                        "Intel Core i5", 3200,
                        "LG 24 Inch", "Logitech", "Wireless");


            pc.displaySpecKomputer();
        }
    }
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ guided ---
Kode Jurusan : DS
Nama Jurusan : Data Science
Daftar Mahasiswa :
1.103132430021 Raymundus Ariel Abas
2.103132430022 Keyla Azzahra
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
LG 24 Inch
Logitech, Wireless
------------------------------------------------
```

C. Unguided

1. Soal Unguided 1

Buatlah program Java berorientasi objek (OOP) menggunakan enkapsulasi (semua *field* private), kata kunci this pada *constructor*, dan array biasa tanpa koleksi/inheritance.

- Tugas 1 (Komposisi):
  - Buat kelas RincianHarga (hargaDasar, pajak 10%, getHargaTotal()).
  - Buat kelas ItemPesanan (namaItem, rincian).
  - *Syarat:* Objek RincianHarga wajib dibuat langsung dengan new di dalam *constructor* ItemPesanan, bukan di Main.
- Tugas 2 (Agregasi):
  - Buat kelas Pesanan (nomorMeja, array daftarItem[], jumlahItem).
  - Sediakan method tambahItem(ItemPesanan item) dan tampilkanPesanan().
  - *Syarat:* Objek ItemPesanan dibuat di Main, lalu dimasukkan satu per satu ke dalam Pesanan.
- Tugas 3 (Asosiasi):
  - Buat kelas Kasir (namaKasir).
  - Sediakan method prosesPembayaran(Pesanan pesanan) untuk menghitung total dan mencetak struk.
  - *Syarat:* Kasir tidak menyimpan objek Pesanan sebagai variabel kelas, melainkan hanya menerimanya lewat parameter method.
- Tugas 4 (Pengujian di Main):
  - Buat kasir "Budi".
  - Buat 3 menu: Americano (18.000), Croissant (25.000), Matcha Latte (22.000).
  - Buat pesanan Meja 4, masukkan ketiga menu tadi, tampilkan daftar pesanan, lalu proses bayar via kasir.

Jawaban:
ItemPesanan.java

```
package raymundusariel.unguided;

public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian;

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return namaItem;
    }

    public RincianHarga getRincian() {
        return rincian;
```

```
        }
}
```

Kasir.java

```
package raymundusariel.unguided;

public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;

        ItemPesanan[] daftarItem = pesanan.getDaftarItem();
        int jumlahItem = pesanan.getJumlahItem();

        for (int i = 0; i < jumlahItem; i++) {
            RincianHarga rincian = daftarItem[i].getRincian();

            totalSubtotal += rincian.getHargaDasar();
            totalPajak += rincian.getPajak();
        }
```

```
        double totalBayar = totalSubtotal + totalPajak;


        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + jumlahItem);
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("================================");
    }
}
```

Pesanan.java

```
package raymundusariel.unguided;
public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;


    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }
```

```
public void tambahItem(ItemPesanan item) {
    if (jumlahItem < daftarItem.length) {
        daftarItem[jumlahItem] = item;
        jumlahItem++;
    } else {
        System.out.println("Pesanan sudah penuh.");
    }
}

public void tampilkanPesanan() {
    System.out.println("=== DAFTAR PESANAN CAFE ===");
    System.out.println("Nomor Meja: " + nomorMeja);

    for (int i = 0; i < jumlahItem; i++) {
        ItemPesanan item = daftarItem[i];
        RincianHarga rincian = item.getRincian();

        System.out.println(
            (i + 1) + ". " + item.getNamaItem()
            + " - Rp " + rincian.getHargaTotal()
            + " (Harga: " + rincian.getHargaDasar()
            + ", Pajak: " + rincian.getPajak() + ")"
        );
    }
```

```
        System.out.println();
    }

    public int getNomorMeja() {
        return nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return daftarItem;
    }

    public int getJumlahItem() {
        return jumlahItem;
    }
}
```

RincianHarga.java

```
package raymundusariel.unguided;
public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
```

```
        this.pajak = hargaDasar * 0.10;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public double getPajak() {
        return pajak;
    }

    public double getHargaTotal() {
        return hargaDasar + pajak;
    }
}
```

Main.java

```
package raymundusariel.unguided;
public class Main {
    public static void main(String[] args) {
        // Membuat objek kasir
        Kasir kasir = new Kasir("Budi");

        // Membuat item pesanan secara independen
        ItemPesanan item1 = new ItemPesanan(
```

```
        "Iced Americano", 18000
    );

    ItemPesanan item2 = new ItemPesanan(
        "Beef Croissant", 25000
    );

    ItemPesanan item3 = new ItemPesanan(
        "Matcha Latte", 22000
    );

    // Membuat pesanan untuk meja 4
    Pesanan pesanan = new Pesanan(4, 5);

    // Memasukkan item ke dalam pesanan
    pesanan.tambahItem(item1);
    pesanan.tambahItem(item2);
    pesanan.tambahItem(item3);

    // Menampilkan daftar pesanan
    pesanan.tampilkanPesanan();

    // Memproses pembayaran
    kasir.prosesPembayaran(pesanan);
}
```

```
}
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ guided ---
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
```

Penjelasan Program

Program Sistem Manajemen Pesanan Cafe dirancang untuk mendokumentasikan pesanan, mengalkulasi beban pajak 10%, serta menangani transaksi pembayaran. Struktur program dibangun melalui lima kelas utama—RincianHarga, ItemPesanan, Pesanan, Kasir, dan Main guna merepresentasikan relasi komposisi, agregasi, dan asosiasi.

Mekanisme kerja program berjalan melalui tahapan berikut:

- Pencatatan Biaya (RincianHarga): Kelas ini memuat atribut hargaDasar serta pajak bertipe double. Nilai pajak ditetapkan secara otomatis sebesar 10% dari harga dasar, dengan metode getHargaTotal() untuk mengembalikan kalkulasi total keduanya.

- Pembentukan Menu (ItemPesanan): Kelas ini memadukan nama menu dengan objek RincianHarga. Penerapan relasi komposisi terwujud saat instansiasi RincianHarga dieksekusi langsung di dalam konstruktor ItemPesanan, menjadikan siklus hidup rincian biaya terikat penuh pada item terkait.

- Pengorganisasian Transaksi (Pesanan): Kelas ini menampung data nomorMeja, array daftarItem, serta pencatat jumlahItem. Hubungan agregasi diimplementasikan melalui metode tambahItem(), yang menerima objek ItemPesanan hasil instansiasi luar untuk disimpan ke dalam array. Informasi seluruh pesanan disajikan melalui pemanggilan metode tampilkanPesanan().

- Penyelesaian Transaksi (Kasir): Relasi asosiasi diterapkan pada metode prosesPembayaran(), di mana objek Pesanan hanya dilewatkan sebagai argumen tanpa dijadikan variabel kelas permanen. Metode ini mengkalkulasi akumulasi subtotal, pajak keseluruhan, hingga nominal akhir tagihan.

- Eksekusi Utama (Main): Alur dimulai dengan membuat instansi kasir "Budi", tiga menu independen (Iced Americano, Beef Croissant, dan Matcha Latte), serta pesanan untuk Meja 4. Seluruh menu dimasukkan ke dalam daftar pesanan, ditampilkan ke layar, lalu diproses hingga mencetak struk dengan subtotal Rp65.000, pajak Rp6.500, dan grand total Rp71.500.

Sistem menerapkan prinsip enkapsulasi ketat dengan menyetel seluruh atribut menjadi private dan menyediakan metode *getter* untuk pembacaan data. Kata kunci this digunakan pada konstruktor guna menghindari kerancuan antara atribut kelas dan parameter. Selain itu, penyimpanan data memanfaatkan array murni berkapasitas tetap tanpa bantuan pustaka koleksi maupun konsep pewarisan (*inheritance*).

D. Kesimpulan

Praktikum ini menegaskan implementasi relasi fundamental antarkelas pada paradigma *Object-Oriented Programming* (OOP) menggunakan Java. Melalui studi kasus Sistem Manajemen Pesanan Cafe, relasi komposisi berhasil diterapkan pada keterikatan kuat antara ItemPesanan dan RincianHarga, agregasi pada pengelolaan array ItemPesanan di dalam wadah Pesanan, serta asosiasi pada keterlibatan fungsional sesaat antara kelas Kasir dan Pesanan. Pemakaian *access modifier* private, penanganan variabel menggunakan this, serta ketiadaan dependensi koleksi dinamis menunjukkan bagaimana modularitas, keamanan data, dan keterhubungan logis antarentitas dapat dibangun secara rapi dan sistematis sesuai rancangan arsitektur perangkat lunak.